
DSEP Group 22

**Lanka Link v3
Master Test Plan**

Version 2.0

Revision History

Date	Version	Description	Author
19/Sep/26	1.0	First master test plan; run on the development laptop against the shared database.	DSEP Group 22
20/Sep/26	2.0	Isolated test stack, mobile layouts, Groq triage and monitoring added; full run, load test on the isolated stack and a read-only load test on production.	DSEP Group 22

Table of Contents

1.	Evaluation Mission and Test Motivation	4
2.	Target Test Items	4
3.	Test Approach	5
3.1	Testing Techniques and Types	6
3.1.1	Data and Database Integrity Testing	6
3.1.2	Function Testing	6
3.1.3	User Interface Testing	8
3.1.4	Performance Profiling	12
3.1.5	Load Testing	13
3.1.6	Security and Access Control Testing	16
3.1.7	Failover and Recovery Testing	17
3.1.8	Configuration Testing	18
3.2	Model Selection and Evaluation	19
3.2.1	Intent and Department Classification	19
3.2.2	Customer Priority: Distillation from Language Model Labels	21
3.2.3	Router Photo Classification	21
3.2.4	Speech Transcription	22
3.2.5	Grounded Response Validation	23
4.	Deliverables	23
4.1	Test Evaluation Summaries	24
4.2	Reporting on Test Coverage	26
5.	Risks, Dependencies, Assumptions, and Constraints	27
6.	References	28

Master Test Plan

1. Evaluation Mission and Test Motivation

This Master Test Plan and Test Report covers Lanka Link v3, a customer support system for a Sri Lankan internet service provider. A customer writes in Sinhala, Tamil, Singlish, Tanglish or English, from a phone or a browser, and may attach a photo of the router or a voice note. The system transcribes the voice note, translates the message into English, reads the customer's line, bill and area record, triages the request, diagnoses the fault, drafts a reply that may only state what the records prove, and puts that reply in front of an agent. The approved reply goes back in the language the customer wrote in.

The system is sixteen services behind one edge, two large models on the request path (NLLB-200 for translation and faster-whisper for speech), language models for triage, diagnosis and drafting (Groq, Ollama Cloud and Gemini), and three stores. Any of those can be slow, wrong or absent, and the product still has to answer. A single wrong amount in a reply, or one customer seeing another customer's ticket, is a failure the business cannot ship.

The mission of this test effort, in priority order:

- **Certify that replies tell the truth.** Every figure a reply quotes (a balance, an outage time) must come from the records and must survive translation unchanged.
- **Certify that the wrong person never sees the wrong data.** Four staff roles and many customers share one database; every surface is checked against every role.
- **Show that the system stays up when a part does not.** Each model and each external language model must be able to fail without the customer losing their message.
- **Measure whether it is fast enough, and where it stops scaling.** Response times for the screens people use, the acknowledgement a customer waits for, and the load at which the pipeline degrades.
- **Verify the changes of this iteration:** the Groq triage models, the mobile layouts and the production monitoring.
- **Record why each model on the path was chosen,** from the offline benchmarks that picked them, so a later change can be compared against the same measurements.
- **Leave the tests behind as a regression fence** that anyone can rerun with one command or one click, without touching production data.

2. Target Test Items

The listing below identifies the software, hardware and supporting elements that are targets for testing.

Item	What it is	Why it is a target
Edge (Caddy) and gateway	Every request enters here; the gateway checks the role on every call	Access control and response time
auth (Better Auth)	Sessions, sign-in throttling, role claims	Security
inquiry	The write a customer waits for; attachments to object storage	Acknowledgement time, data integrity
orchestrator	Stage budgets, revisions, the case record	Failover, performance
triage (new: Groq)	Department rules; customer priority from a Groq model or the distilled TriageModel	Function, failover under rate limits
knowledge, grounding, response	Diagnosis, the evidence bundle, the drafted reply	Truthfulness, load capacity
translation, audio, image	NLLB-200, faster-whisper, the router photo classifier	Numbers across languages, resilience
business, payments, control	The ledger, payments stub, live model bindings	Data integrity, hot model swap

web (new: mobile layouts)	Every screen for customers, agents, admins and operators	User interface on desktop and phone
Neon Postgres, TimescaleDB, MinIO, NATS, Valkey	Third-party stores and messaging	Durability, round-trip cost
Monitoring (new)	Prometheus, Grafana, node-exporter, cAdvisor, blackbox probes	Operability of the VPS
The models themselves	Intent and priority text models, the router photo classifier, speech, the drafting design	Chosen by benchmark (section 3.2), and re-checked live by the stage verifications
Deployment	GitHub Actions, GHCR images, Ansible, the Ubuntu VPS	A push to main must reach production

Out of scope for this iteration: Firefox and Safari (Chromium only, desktop and a 375 pixel phone profile), a commissioned penetration test, and the MLOps profile (Airflow, MLflow).

3. Test Approach

Everything is tested through the interfaces a real user or operator has: the HTTP edge and the browser. A passing suite then means the product works, not that a function does. Tests are black box, automated and self-verifying: each one decides pass or fail from status codes, ids, digits and scripts, never from a person reading a screen.

The test environment is isolated. Development and production share one Neon database, so running tests against the normal stack would write test tickets, suspend accounts and push load into production. This iteration adds `compose.test.yaml`, which runs the complete stack (all sixteen services and both models) on a throwaway Postgres built from scratch by the same migration and seed that production uses. Production itself is only exercised read-only, by the second load test.

Level	What it covers	Command	Needs
Unit and contract	Policy, fusion, translation guards, triage fallback, architecture rules	<code>bash scripts/test-all.sh</code>	Nothing
System	Function, access control, data integrity, performance, configuration, failover	<code>pytest tests/system</code>	Test stack
Browser	Screens on desktop Chromium and a 375 px phone	<code>npx playwright test</code>	Test stack
Scenario	The headline Sinhala case end to end; live model swap	<code>demo_flow.py</code> , <code>verify_hot_swap.py</code>	Test stack
Load	Write path at 5 inquiries a second; read-only browsing on production	<code>scripts/load.sh</code>	Test stack, production
Everything	All of the above, evidence kept	<code>bash scripts/test-plan.sh</code>	Docker

Reproducible and in version control (GitOps). The plan is code: the suites, the stack definition, the load scenarios and the runner live in the repository. `bash scripts/test-plan.sh` runs the whole plan on any machine with Docker and writes every log, JUnit XML file, k6 summary and screenshot into `reports/<date>/. The same runner is a GitHub Actions workflow (test-plan.yml, run from the Actions tab or weekly) that uploads that folder as an artifact. The CI end-to-end job now uses the same isolated stack on every push, so it no longer waits for a separate database. The logs, JUnit XML, k6 summaries and screenshots this report quotes are kept beside it in docs/test-report/evidence, so any number here can be checked against the run that produced it.`

3.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

The databases and the processes that write them are tested as a subsystem, through the one service that owns each store, since no test (and no other service) may reach a store directly.

Technique Objective:	Show that a write is durable and reads back exactly as written, that a bad write is refused, and that an unknown id gives a clean 404 rather than a leak or a 500.
Technique:	• Write a message with Sinhala text, an amount and a time; read it back from the conversation and compare character for character. • Upload an attachment and fetch it back (object storage round trip). • Check that each invoice's displayed total is the stored total formatted, so no screen recomputes money. • Send an empty message and unknown ids to every store-backed route. • Check that a case revision only moves forward. • New: build the whole database from nothing (migrate and seed on an empty Postgres) on every run.
Oracles:	Self-verifying: the value written is the expected value, so the assertion is equality. For money the oracle is the ledger row, never a second calculation of the same figure.
Required Tools:	• pytest and httpx (tests/system/test_data_integrity.py) • compose.test.yaml (Postgres with pgvector, built from scratch) • psql inside the test database for diagnosis
Success Criteria:	Every store-backed route has a round trip and a rejection test, and all pass. A fresh install starts without manual steps.
Special Considerations:	• Tests only append, so one database can serve many runs. The duplicate guard refuses the same text twice within ten minutes, so every test message carries a timestamp.

Result: 8 of 8 passed. Building the database from nothing found two defects that the shared, long-lived database had hidden for weeks: the inquiry service could not start on an empty database because a backfill statement read a table that is only created later in the same script (D10), and the seed forced TLS on every database connection (D11). Both were fixed; an empty database now comes up with no manual step.

```
$ uv run pytest tests/system/test_data_integrity.py -v (from the full run, 8 results)
tests/system/test_data_integrity.py::test_an_unknown_id_is_a_clean_404[/api/app/tickets/LL-999999] PASSED [ 50%]
tests/system/test_data_integrity.py::test_an_unknown_id_is_a_clean_404[/api/app/orders/00000000-0000-0000-0000-000000000000] PASSED [ 52%]
tests/system/test_data_integrity.py::test_an_unknown_case_is_a_clean_404 PASSED [ 54%]
tests/system/test_data_integrity.py::test_an_empty_message_is_refused PASSED [ 56%]
tests/system/test_data_integrity.py::test_a_message_survives_the_round_trip PASSED [ 58%]
tests/system/test_data_integrity.py::test_an_attachment_keeps_its_bytes PASSED [ 60%]
tests/system/test_data_integrity.py::test_a_case_revision_only_moves_forward PASSED [ 62%]
tests/system/test_data_integrity.py::test_the_bill_adds_up PASSED [ 64%]
```

Listing 1: Data and database integrity tests, from the full run.

3.1.2 Function Testing

Function testing follows the use cases, driven through the same API the browser calls, by the persona who owns each one.

Technique Objective:	Exercise each use case with valid and invalid input and confirm the business rules, including the headline case: a suspended customer inside an outage writes in Sinhala and must get a truthful Sinhala reply.
----------------------	---

Technique:	- Public: plans without an account. Customer: overview, billing, usage, plan, notices, tickets. - Headline case (demo_flow.py --strict): suspend Ravi's account, send his Sinhala message, wait for the draft, require it to cite both the unpaid balance and the outage, approve it, and require the Sinhala reply to carry the same amount. - Staff: the queue, a case with its evidence, approval, a language override. - Admin: rebind the drafting model live and back (verify_hot_swap.py). - New: triage through Groq, checked by calling the triage service and by the unit tests of its fallback.
Oracles:	Status codes, ids, the script of the reply (Sinhala characters present), and number preservation: the digits in the English draft must equal the digits in the translated reply. Wording is not machine-checkable and is read by a person.
Required Tools:	- pytest, httpx - scripts/demo_flow.py, scripts/verify_hot_swap.py - Real models: Ollama Cloud gpt-oss for drafting and diagnosis, Groq for triage, NLLB-200
Success Criteria:	Every use case passes, and the headline case passes end to end with real models on the path, not the offline stub.
Special Considerations:	The pipeline is asynchronous: the acknowledgement returns at once and the draft arrives 15 to 40 seconds later, so tests poll with a budget instead of sleeping.

Result: 6 of 6 function tests passed; the headline scenario and the live model swap passed. Ravi's Sinhala message was acknowledged in about 60 ms and drafted in under 30 s. The draft cited both the unpaid balance and the outage, and the Sinhala reply carried the amount unchanged (රුපියල් 18,574.40). Reading the Sinhala reply found one wording defect: a conjunct split by a space ("මධ්‍යම" for "මධ්‍යම", central), left open as D13.

```
$ uv run --with httpx python scripts/demo_flow.py --strict
1. suspend_account on SUB-100002: 422 (already suspended)
2. Ravi wrote in Sinhala, acknowledged in 139 ms
3. LL-10312 drafted (held) after 24 s:
   Ravi, you said the internet has been down for two days and the internet LED is blinking irregularly.
   Because we have not received a photo of your router, we cannot identify the exact model to give you specific
   wiring steps. Please attach a picture of the back of the unit so we can guide you further. Our records show an
   active incident on the Colombo Central feeder that has cut the fibre serving your line, and the line is
   currently marked as down. In addition, your account is suspended due to an unpaid balance of LKR 18,574.40,
   which also prevents service from being delivered. Both issues must be resolved before the connection can be
   restored. Please settle the outstanding balance of LKR 18,574.40; once the payment is processed the suspension
   will be lifted and the service will be restored as soon as the network repair is completed.

Lanka Link customer support
  cites payment: True, cites outage: True
4. agent approved
   Ravi received after 29 s (Sinhala):
   රවි, ඔයා කිව්වා ඉන්ටර්නෙට් අක්‍රීය වෙලා දවස් දෙකයි කියලා. ඉන්ටර්නෙට් LED එක අක්‍රමිකව බැලුවා කියලා. අපිට ඔයාගේ
   රවුටරයේ ඡායාරූපයක් ලැබ්ලා නැති නිත්දා අපිට හරියටම මොඩලය හඳුනාගන්න බැහැ. කරුණාකර ඒකකයේ පිටුපස
   ඡායාරූපයක් අමුණන්න එව්ට අපට තව දුරටත් ඔබට මග පෙන්විය හැකියි. අපේ වාර්තා පෙන්නුම් කරනවා කොලොම්බෝ සෙන්ට්‍රල්
   රජ් ෆීඩර් එකේ ක්‍රියාකාරී සිද්ධියක් වෙලා තියෙනවා ඒකෙන් ඔයාගේ රේඛාවට සේවය කරන ෆයිබර් එක කපලා. මීට අමතරව,
   රුපියල් 18,574.40 ක නොගෙවූ ශේෂයක් හේතුවෙන් ඔබගේ ගිණුම අත්හිටුවන අතර එමඟින් සේවාව ලබා දීම වළක්වයි.
   සම්බන්ධතාවය යථා තත්ත්වයට පත් කරන්න කලින් ප්‍රශ්න දෙකම විසඳිය යුතුයි. කරුණාකර රුපියල් 18,574.40 ක ශේෂය විසඳන්න;
   ගෙවීම සකස් කළ පසු අත්හිටුවීම ඉවත් කරනු ඇති අතර ජාල අවන්වැසියා කිරීම අවසන් වූ වහාම සේවාව නැවත ආරම්භ කරනු
   ඇත.

Lanka Link customer support
demo flow OK
```

Listing 2: The headline case end to end, with a real model and a Sinhala reply.

Groq triage. The new `llm_triage` binding scores the customer side priority with `qwen/qwen3.8-27b` on Groq, with the distilled `TriageModel` as the fallback. Measured directly: about 240 ms a call, and the same message always gets the same level once temperature is 0 (eight calls, eight identical levels; with the default temperature the same message scored 3, 8, 5 and 7). Five Groq models were checked for valid JSON and latency before choosing the default:

Groq model	JSON valid	Latency (two calls)	Choice
qwen/qwen3.8-27b	Yes	299 ms, 282 ms	Default: fastest and steady
openai/gpt-oss-20b	Yes	702 ms, 512 ms	Selectable
openai/gpt-oss-120b	Yes	694 ms, 566 ms	Selectable
groq/compound-mini	Yes	989 ms, 861 ms	Selectable
groq/compound	Yes	not measured	Selectable

```
$ uv run --with httpx python scripts/verify_hot_swap.py
before: ollama:gpt-oss:120b-cloud generation 3
swapped: stub probes reachable in 1 ms
restored: ollama:gpt-oss:120b-cloud generation 5
hot swap OK
```

Listing 3: The drafting model is rebound live, answers, and is restored.

3.1.3 User Interface Testing

User interface testing checks that each screen shows the right data to the role that opened it, that navigation goes where it says, that nothing from the back office reaches a customer screen, and, new in this iteration, that every screen works on a phone.

Technique Objective:	Exercise every screen on desktop Chromium and on a 375 pixel wide phone, observing navigation, role gates, the customer immersion rule and layout.
Technique:	- Sign in as each role; confirm where it lands and what it cannot reach. - Console: open the queue and a case with its draft. Admin: open the model map and a stage. - Immersion: walk every customer page and fail on any back office word (subscriber ids, OLT, LLM, triage, confidence, null, undefined, NaN, the em dash). - Session: a reload keeps the session; signing out ends it and lands on the website. - New, mobile: visit 26 pages across all roles at 375 x 812 with touch, fail if any page scrolls sideways (naming the element that sticks out), and keep a screenshot of each.
Oracles:	Roles and accessible names rather than CSS selectors, so a test breaks when meaning changes and not when styling does. The mobile oracle is geometric: the page's scroll width must not exceed the viewport. Visual quality is judged by a person from the screenshots.
Required Tools:	- Playwright with Chromium (web/e2e), a Pixel 7 touch profile narrowed to 375 px - Screenshots kept per page for review
Success Criteria:	Every major screen is opened by a test on both form factors, and none scrolls sideways.
Special Considerations:	The live update stream never goes idle, so tests wait for content, not for network silence. Sign-in is rate limited per address, so the helper backs off.

Result: 11 desktop and 5 mobile browser tests passed, 0 failed. The first mobile audit, run against the build that preceded this iteration, found four pages that scrolled sideways (the landing page, plans, documentation and the incidents page) and, from the screenshots, three layouts that fitted but did not work on a phone: the customer menu hid four of its six items off screen, the agent inbox table cut off everything after the summary, and the model map shrank to unreadable size. The fixes: a bottom tab bar for customers (five tabs, 56 px targets, safe area aware) with settings in the header, the inbox as cards, the model map as a list on phones, a sim clock bar that scrolls within itself, and the overflow causes themselves (a header whose hidden class lost to the button's own display, a grid child without min-width, oversize key caps). The run also found D12: signing out of the customer area landed on the sign-in page instead of the website.

The screenshots below are the product in dark appearance, which is what a phone or a desktop set to dark shows; the suites capture both form factors on every run. The first set is the build that preceded this iteration, photographed by the same suite, which also fails it: three of its pages scroll sideways.

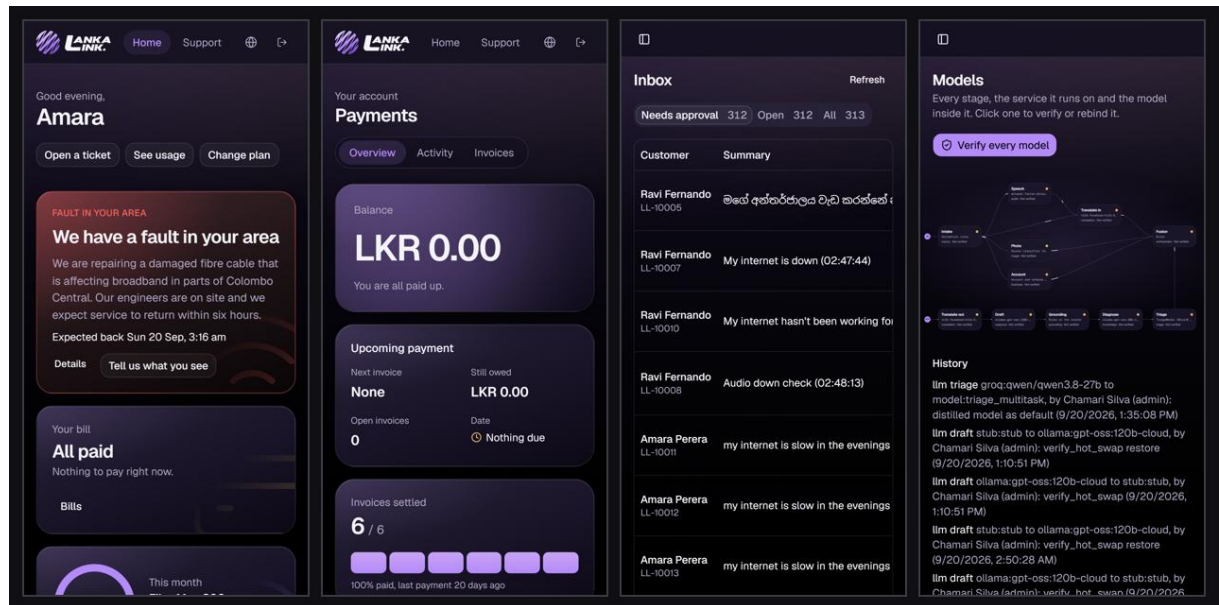


Figure 4: Before the changes: the customer menu shows two of its six items, the inbox is cut off at the summary, and the model map has shrunk to unreadable size.

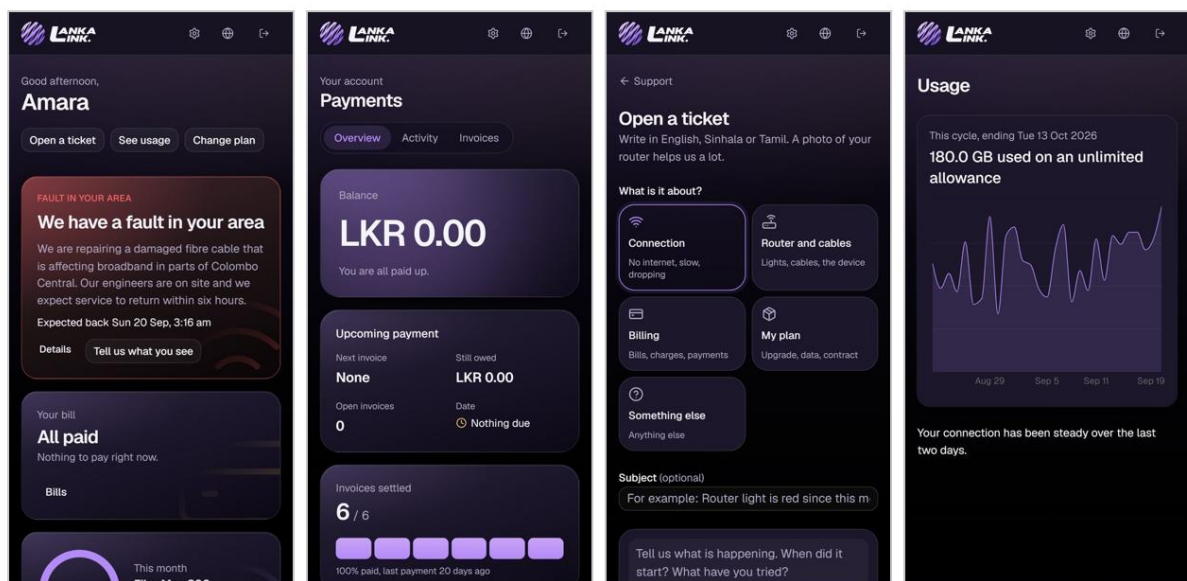


Figure 5: After, customer screens at 375 px, with the bottom tab bar.

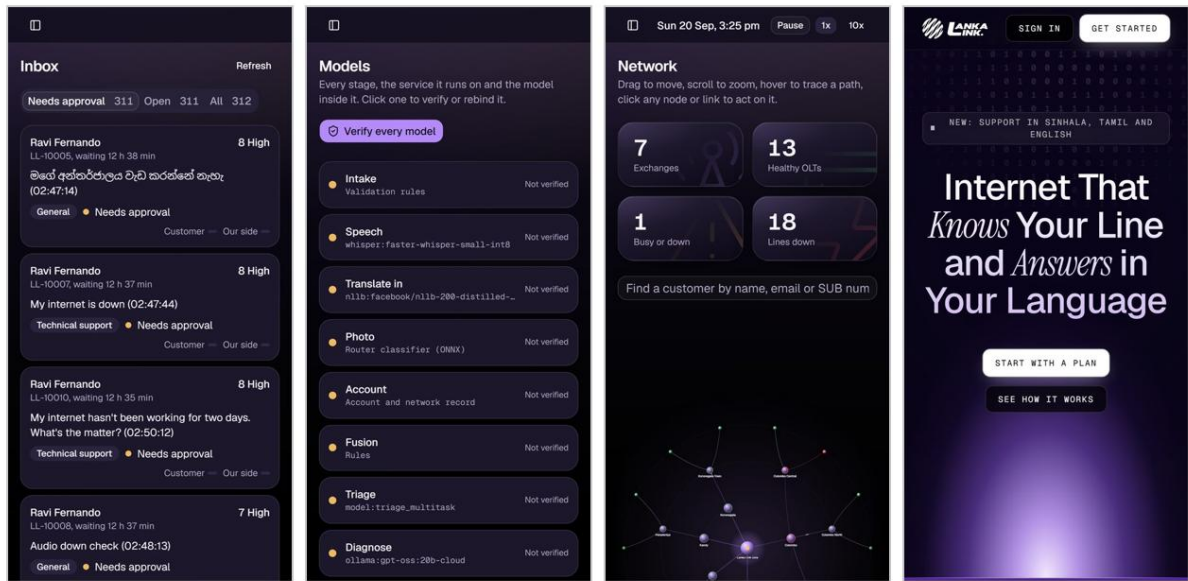


Figure 6: After, the agent inbox as cards, the model map as a list, the network map, the landing page.

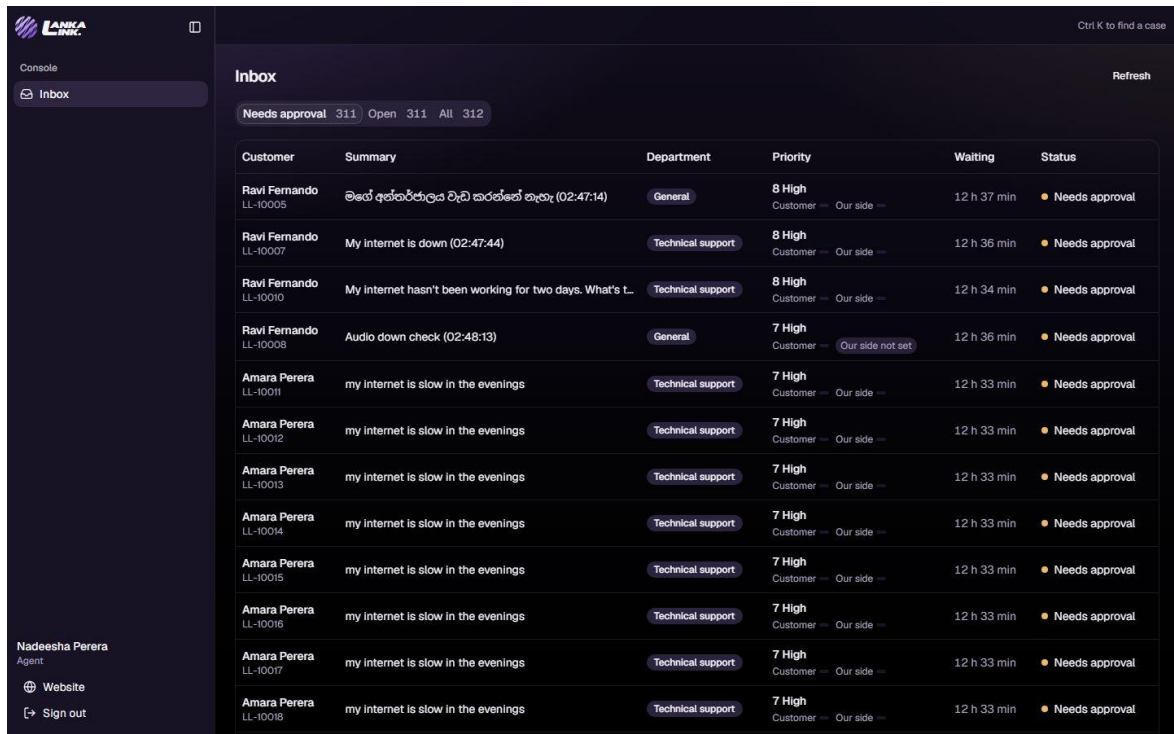


Figure 7: The agent console on a desktop browser, dark.

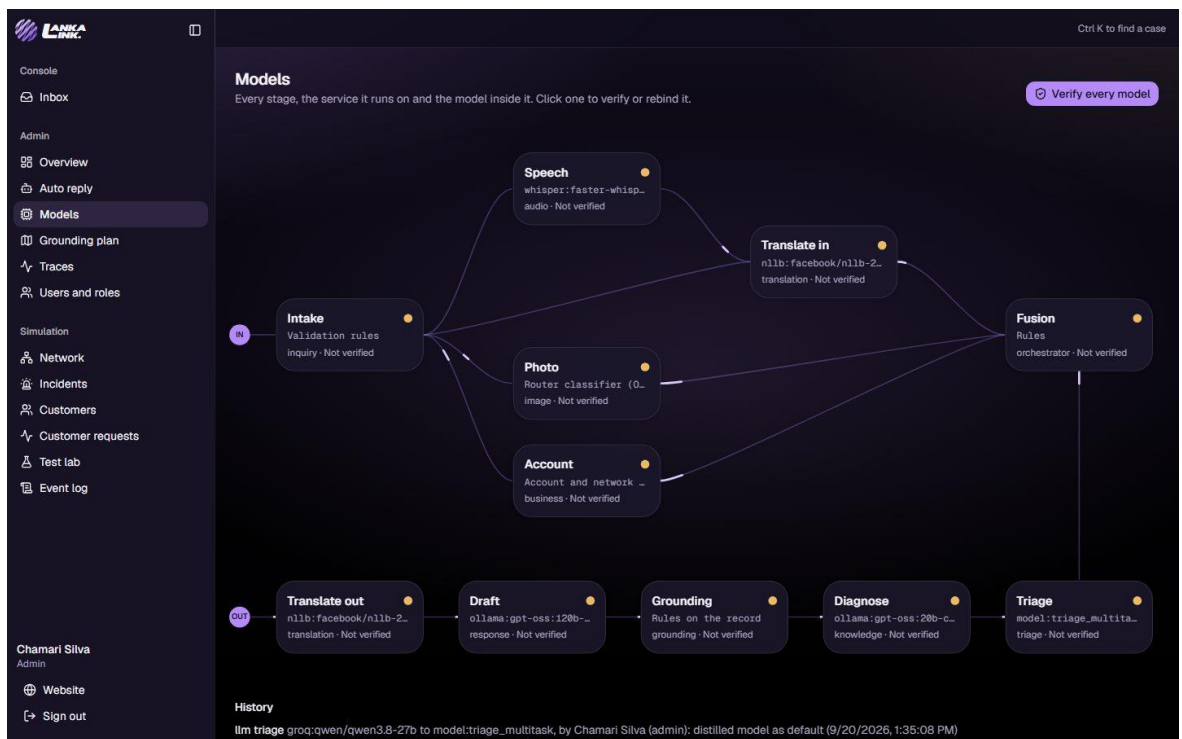


Figure 8: The model map, where every stage is verified and rebound.

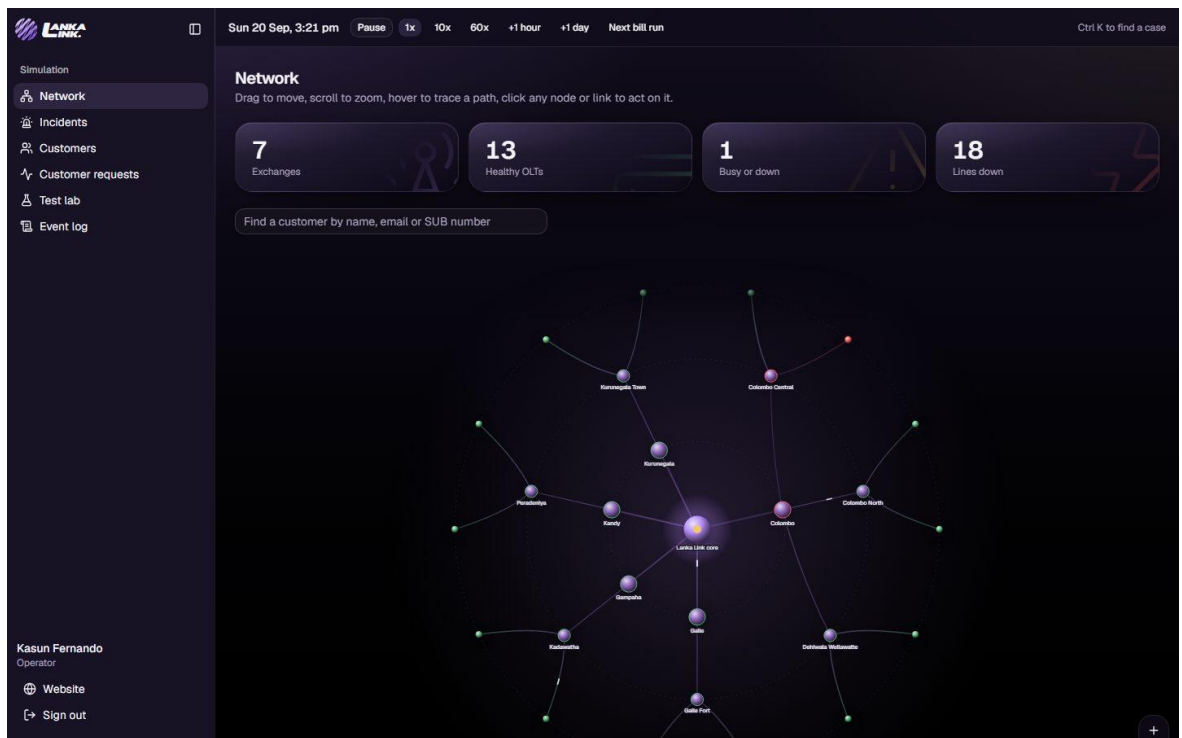


Figure 9: The simulated network an operator breaks things on.

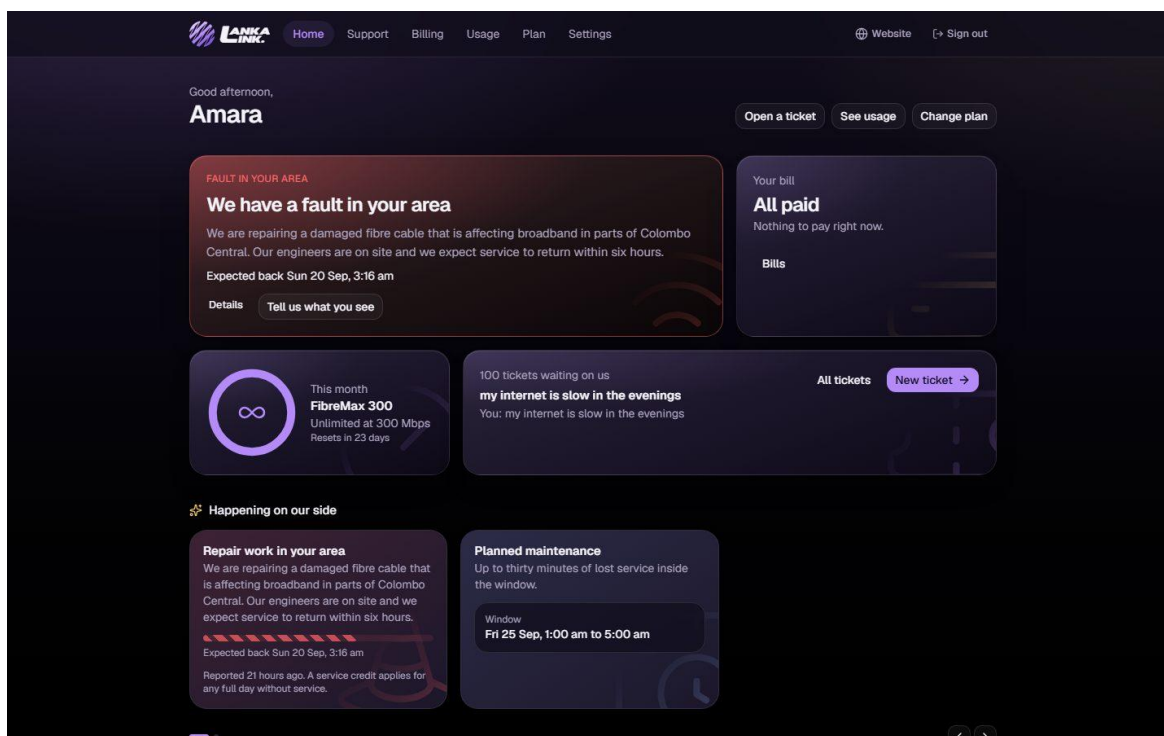


Figure 10: The customer's home screen on a desktop browser.

```
$ bash -c cd web && SHOTS_DIR='/d/DSEP22/v3/reports/2026-09-20-final/screens/mobile'
PLAYWRIGHT_JUNIT_OUTPUT_NAME='/d/DSEP22/v3/reports/2026-09-20-final/junit-browser.xml' npx playwright test --
reporter=list,junit
```

Running 16 tests using 4 workers

```
ok 3 [chromium] > e2e\staff.spec.ts:5:5 > the console queue opens a case with its draft and its evidence
(3.3s)
ok 2 [chromium] > e2e\auth.spec.ts:5:5 > an agent lands in the console and cannot open admin or sim (3.8s)
ok 1 [chromium] > e2e\customer.spec.ts:7:5 > immersion: no customer page shows back office words (12.7s)
ok 7 [chromium] > e2e\customer.spec.ts:20:5 > opening a ticket is acknowledged and shows progress (1.1s)
ok 4 [mobile] > e2e\mobile.spec.ts:33:5 > public pages fit a phone (15.1s)
ok 5 [chromium] > e2e\staff.spec.ts:17:5 > the models map draws every stage and opens one (14.8s)
ok 8 [mobile] > e2e\mobile.spec.ts:39:5 > customer pages fit a phone (16.8s)
ok 9 [chromium] > e2e\staff.spec.ts:27:5 > a customer is kept out of the staff areas (14.6s)
ok 11 [chromium] > e2e\staff.spec.ts:35:5 > the session survives a reload and shows on the landing page
(886ms)
ok 10 [mobile] > e2e\mobile.spec.ts:52:5 > console fits a phone (4.8s)
ok 12 [chromium] > e2e\staff.spec.ts:43:5 > signing out ends the session (14.2s)
ok 6 [chromium] > e2e\auth.spec.ts:14:5 > an operator lands in the simulation panel (49.5s)
ok 14 [chromium] > e2e\auth.spec.ts:20:5 > a wrong password is refused in plain words (327ms)
ok 15 [chromium] > e2e\auth.spec.ts:28:5 > sign up with email reaches onboarding (596ms)
ok 13 [mobile] > e2e\mobile.spec.ts:64:5 > admin fits a phone (24.6s)
ok 16 [mobile] > e2e\mobile.spec.ts:72:5 > simulation fits a phone (25.5s)

16 passed (1.5m)
```

Listing 4: Browser tests: the desktop, mobile and screenshot projects.

3.1.4 Performance Profiling

Performance profiling measures the latencies a person waits for, on a single user, and profiles where time goes inside the pipeline.

Technique Objective:

Measure the acknowledgement after pressing send, the screens an agent and a customer open repeatedly, and each pipeline stage, under a normal and a heavy workload.

Technique:	• Single user, warm connection, five samples per route, median against a budget. • Every stage of every case records its duration and outcome (cases.stage_event), which is summarised per stage after the load run. • On production: the Grafana dashboard's edge latency (p50, p95, p99) and host resources.
Oracles:	Budgets set above the best measurement and below the point where a screen feels slow, so a regression fails without noise failing it. The measured median is printed on failure.
Required Tools:	• pytest (tests/system/test_performance.py) • SQL over cases.stage_event • Prometheus and Grafana
Success Criteria:	Every budget met on a warm stack; every stage's p95 inside its budget at normal load.
Special Considerations:	• The isolated test database is on the same machine, so its numbers show the code, not the network. Production numbers, with the database in Singapore and the server in Malaysia, come from the read-only load test.

Result: 8 of 8 budgets met.

Where the time goes, per stage, over every case of the run (including the load burst):

Stage	Outcome	Cases	p50	p95
Translate in	done	311	5 ms	8 ms
Account prefetch	done	312	6 ms	14 ms
Triage	done	312	20 ms	282 ms
Diagnose	done	22	12.7 s	14.8 s
Diagnose	budget missed	290	15.0 s	15.0 s
Grounding	done	311	9 ms	18 ms
Draft	done	26	16.8 s	55.6 s
Draft	budget missed	285	60.0 s	60.0 s

The local stages (translation, account prefetch, grounding) take milliseconds. The three stages that call an external language model (triage, diagnosis, drafting) dominate, and they are where capacity ends, as the load test shows.

```
$ uv run pytest tests/system/test_performance.py -v (from the full run, 8 results)
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/me-800] PASSED [ 78%]
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/app/overview-1500] PASSED [ 80%]
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/app/billing-800] PASSED [ 82%]
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/app/plan-800] PASSED [ 84%]
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/app/usage-1200] PASSED [ 86%]
tests/system/test_performance.py::test_a_warm_account_screen_is_within_budget[/api/app/notices-800] PASSED [ 88%]
tests/system/test_performance.py::test_the_acknowledgement_is_immediate PASSED [ 90%]
tests/system/test_performance.py::test_the_console_queue_loads_within_budget PASSED [ 92%]
```

Listing 5: Performance budgets, from the full run.

3.1.5 Load Testing

Load testing subjects the system to sustained and rising workloads, on the write path in isolation and on the read path in production.

Technique Objective:	Show that the system holds a sustained arrival rate without errors, find where it degrades, and measure production under concurrent browsing.
Technique:	• Write path (isolated stack): k6 at a constant 5 new tickets a second for one minute, up to 20 virtual users, each iteration sending a message and reading the account overview. That is 300 tickets a minute, far above expected traffic, and every one runs the full pipeline. • Read path (production): k6 ramping to 20 concurrent users over one minute, holding three minutes, ramping down over one; each user loads the landing page, the plan catalogue and the plans page, then a signed-in customer's overview, billing and tickets. No writes. • Production is watched on the Grafana dashboard during the run.
Oracles:	k6 thresholds, which fail the run by exit code: acknowledgement p50 under 300 ms and p95 under 800 ms, overview p50 under 250 ms, page p95 under 2 s, API p95 under 1.5 s, failures under 1 percent.
Required Tools:	• k6 (scripts/load.sh; load/ack.js, load/browse.js) • Grafana, Prometheus, cAdvisor
Success Criteria:	Zero failed requests and every threshold met; the degradation point identified and shown to fail safe.
Special Considerations:	• Write load is never run against production: it would open hundreds of real tickets. The production run signs in once, since sign-in is rate limited per address by design.

Write path, isolated stack:

Metric	Result	Threshold	Verdict
Requests	604, 0 failed	under 1 percent failed	Pass
Tickets opened	301 in 60 s		
Acknowledgement p50	9.7 ms	300 ms	Pass
Acknowledgement p95	13 ms	800 ms	Pass
Overview p50	4.8 ms	250 ms	Pass

```
$ env SUMMARY=reports/2026-09-20-final/load-ack.json bash scripts/load.sh
```



```

execution: local
script: /work/load/ack.js
output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m30s max duration (incl. graceful stop):
    * chat: 5.00 iterations/s for 1m0s (maxVUs: 20, gracefulStop: 30s)

running (0m01.0s), 00/20 VUs, 5 complete and 0 interrupted iterations
chat [ 1% ] 00/20 VUs 0m00.9s/1m0s 5.00 iters/s

running (0m02.0s), 00/20 VUs, 10 complete and 0 interrupted iterations
chat [ 3% ] 00/20 VUs 0m01.9s/1m0s 5.00 iters/s

running (0m03.0s), 00/20 VUs, 15 complete and 0 interrupted iterations
chat [ 5% ] 00/20 VUs 0m02.9s/1m0s 5.00 iters/s
... 206 lines omitted ...
iterations.....: 301      5.006217/s
vus.....: 0      min=0      max=1
vus_max.....: 20      min=20      max=20

NETWORK
data_received.....: 492 kB 8.2 kB/s
data_sent.....: 378 kB 6.3 kB/s

running (1m00.1s), 00/20 VUs, 301 complete and 0 interrupted iterations
chat ✓ [ 100% ] 00/20 VUs 1m0s 5.00 iters/s

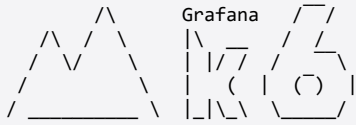
```

Listing 6: k6 on the write path: every threshold met, no failed request.

Behind the acknowledgement, the pipeline under that burst. Every one of the 312 cases was created, triaged and put in front of an agent. Triage used Groq for 45 of them (average 315 ms); Groq's free tier allows 8,000 tokens a minute, so the other 267 fell back to the distilled TriageModel as designed, with no failure. Diagnosis and drafting call Ollama Cloud, which accepts four requests at a time per client: 290 diagnoses reached their 15 s budget and 285 drafts their 30 s budget, so those cases reached the agent without a proposed reply. That is the system's capacity limit: about fifteen drafts a minute with the current model account. It fails safe (no message is lost, no customer sees an error), and it is far above today's traffic, but it is the first thing to scale: a paid model tier, Groq for drafting, or a queue that drafts in arrival order.

Read path, production (<https://172-197-200-110.sslip.io>, 2 vCPU, 7.7 GB, database in Singapore):

Metric	Result	Threshold	Verdict
Requests	5348, 0 failed (17.6 a second)	under 1 percent failed	Pass
Concurrent users	20 (ramped)		
Page p50 / p95	513 ms / 1,809 ms	p95 under 2 s	Pass
API p50 / p95	191 ms / 698 ms	p95 under 1.5 s	Pass



```
execution: local
  script: /work/load/browse.js
  output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 5m30s max duration (incl. graceful stop):
  * browse: Up to 20 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (0m01.0s), 01/20 VUs, 0 complete and 0 interrupted iterations
browse [ 0% ] 01/20 VUs 0m00.1s/5m00.0s

running (0m02.0s), 01/20 VUs, 0 complete and 0 interrupted iterations
browse [ 0% ] 01/20 VUs 0m01.1s/5m00.0s

running (0m03.0s), 01/20 VUs, 0 complete and 0 interrupted iterations
browse [ 1% ] 01/20 VUs 0m02.1s/5m00.0s

... 940 lines omitted ...
iterations.....: 891      2.927195/s
vus.....: 1      min=1      max=20
vus_max.....: 20      min=20      max=20

NETWORK
data_received.....: 160 MB 527 kB/s
data_sent.....: 749 kB 2.5 kB/s

running (5m04.4s), 00/20 VUs, 891 complete and 0 interrupted iterations
browse ✓ [ 100% ] 00/20 VUs 5m0s
```

Listing 7: k6 against production, read only.

The server held 20 concurrent readers with no failed request while it was also running its own simulated traffic and the monitoring stack, on 2 vCPU and 7.9 GB. CPU sat near 49 percent and memory at 53 percent during the run, and the page p95 of 1.8 s is mostly the landing page, which is rendered per request. Writes were deliberately not load tested against production.

3.1.6 Security and Access Control Testing

Security testing covers application-level security (each role reaches only its own functions and data) and system-level security (sessions cannot be forged, secrets cannot leak, the servers expose only what they must).

Technique Objective:	Application level: no role reaches a surface that is not its own, and no customer reaches another customer's row. System level: a session cannot be forged, secrets never reach the repository or the server beyond a whitelist.
----------------------	--

Technique:	• Every staff route with no token: expect 401. With a customer token: expect 403. Admin-only routes with an agent token: expect 403. Each role on its own surface: expect 200. • Two customers fetch each other's ticket by id: expect 403 or 404, never the row. New: on a fresh database the test opens the tickets it needs rather than skipping. • A token with an altered signature: 401. A wrong password: refused. • gitleaks over the whole history on every push; the server's .env is rendered from a whitelist. • New: Grafana has no anonymous access and refuses to start without an admin password; Prometheus and the exporters are not published outside the compose network.
Oracles:	The status code is the assertion; a test fails if the body of a forbidden row ever arrives.
Required Tools:	• pytest (tests/system/test_access_control.py) • gitleaks in CI • Grafana login check
Success Criteria:	Every staff and admin route is covered for the unauthenticated and the wrong-role case, and all pass.
Special Considerations:	• The browser role gate is a convenience; the gateway is the control, so the tests assert against the API.

Result: 15 of 15 passed. In the first run of this iteration the tenant isolation test was skipped, because on an empty database neither persona had a ticket yet; a security test that silently skips on every clean install is a hole, so it now creates its own tickets. The seeded demo passwords are public in the repository and still active on the public site: recorded as a risk in section 5.

```
$ uv run pytest tests/system/test_access_control.py -v (from the full run, 15 results)
tests/system/test_access_control.py::test_no_token_is_rejected[/api/me] PASSED [ 1%]
tests/system/test_access_control.py::test_no_token_is_rejected[/api/app/billing] PASSED [ 3%]
tests/system/test_access_control.py::test_no_token_is_rejected[/api/console/cases] PASSED [ 5%]
tests/system/test_access_control.py::test_no_token_is_rejected[/api/admin/models] PASSED [ 7%]
tests/system/test_access_control.py::test_a_customer_cannot_reach_staff_surfaces[/api/console/cases] PASSED [ 9%]
tests/system/test_access_control.py::test_a_customer_cannot_reach_staff_surfaces[/api/console/ping] PASSED [ 11%]
tests/system/test_access_control.py::test_a_customer_cannot_reach_staff_surfaces[/api/admin/models] PASSED [ 13%]
tests/system/test_access_control.py::test_a_customer_cannot_reach_staff_surfaces[/api/lab/requests] PASSED [ 15%]
tests/system/test_access_control.py::test_an_agent_cannot_reach_admin_surfaces[/api/admin/models] PASSED [ 17%]
tests/system/test_access_control.py::test_an_agent_cannot_reach_admin_surfaces[/api/admin/traces] PASSED [ 19%]
tests/system/test_access_control.py::test_an_agent_cannot_reach_admin_surfaces[/api/lab/requests] PASSED [ 21%]
tests/system/test_access_control.py::test_each_role_reaches_its_own_surface PASSED [ 23%]
tests/system/test_access_control.py::test_a_customer_cannot_read_another_customers_ticket PASSED [ 25%]
tests/system/test_access_control.py::test_a_tampered_token_is_rejected PASSED [ 27%]
tests/system/test_access_control.py::test_sign_in_rejects_a_wrong_password PASSED [ 29%]
```

Listing 8: Access control tests, from the full run.

3.1.7 Failover and Recovery Testing

Failover and recovery testing shows that each optional part can fail without the customer losing their message, and that a restarted part rejoins on its own.

Technique Objective:	Simulate the failure of a model service and of an external model, and exercise recovery, observing that no message is lost and no manual step is needed.
----------------------	--

Technique:	• Stop translation, send a real message: the case must still open and reach a draft. • Stop audio: the message is still accepted. Stop grounding: the model map reports it down rather than healthy. Restart translation: it rejoins with no other action. • New: the triage language model fails, times out (2 s) or is rate limited: the distilled TriageModel answers instead, and the case says why. Unit tests force each path; the load burst forced it for real. • The drafting model: a fallback binding and a circuit breaker, proven by the live swap.
Oracles:	The case exists, the draft exists, the map's status field, and the priority's source field.
Required Tools:	• pytest with LANKA_RESILIENCE=1 (tests/system/test_resilience.py), which stops and restarts containers of the test stack only • services/triage/tests/test_llm_triage.py
Success Criteria:	No message lost in any single-service outage; recovery needs no manual step.
Special Considerations:	• These tests stop containers, so they run only on the isolated stack, never on production.

Result: 4 of 4 passed. Under the load burst the triage fallback carried 267 cases with no triage failure.

```
$ uv run pytest tests/system/test_resilience.py -v (from the full run, 4 results)
tests/system/test_resilience.py::test_a_dead_translation_service_still_answers_the_customer PASSED [ 94%]
tests/system/test_resilience.py::test_a_dead_audio_service_does_not_lose_the_ticket PASSED [ 96%]
tests/system/test_resilience.py::test_the_edge_reports_a_stopped_dependency PASSED [ 98%]
tests/system/test_resilience.py::test_a_restarted_service_is_healthy_again PASSED [100%]
```

Listing 9: Failover and recovery tests, from the full run.

3.1.8 Configuration Testing

Configuration testing verifies every deployment shape the project ships before anyone tries to run it.

Technique Objective:	Show that each combination of compose files is valid and complete, and that the system installs from nothing.
Technique:	• <code>docker compose config</code> for each shape: base, lite, CI, GPU, production, and new: test, CI with test, production with monitoring. • The MLOps profile adds Airflow and MLflow. • Every variable any compose file reads must exist in <code>.env.example</code>. • New: the full stack is built from an empty database on every run (<code>compose.test.yaml</code>).
Oracles:	Exit codes and a set difference of variable names.
Required Tools:	• pytest (tests/system/test_configuration.py) • Docker Compose
Success Criteria:	All shapes valid, no variable missing, a clean install starts.
Special Considerations:	• Production refuses to render without a host name and the monitoring overlay without a Grafana password; both are deliberate.

Result: 10 of 10 passed. The clean install is also where D10 and D11 were found.

```
$ uv run pytest tests/system/test_configuration.py -v (from the full run, 10 results)
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml] PASSED [ 31%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+lite] PASSED [ 33%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+ci] PASSED [ 35%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+gpu] PASSED [ 37%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+prod] PASSED [ 39%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+test] PASSED [ 41%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+ci+test] PASSED [ 43%]
tests/system/test_configuration.py::test_the_compose_file_is_valid[yaml+prod+monitoring] PASSED [ 45%]
tests/system/test_configuration.py::test_the_mlops_profile_adds_airflow_and_mlflow PASSED [ 47%]
tests/system/test_configuration.py::test_every_variable_the_compose_file_reads_is_in_the_example PASSED [ 49%]
```

Listing 10: Configuration tests.

3.2 Model Selection and Evaluation

Every model on the request path was chosen by benchmark before the system was built, not by reputation. The work is in the repository as runnable notebooks with a shared library, a fixed held-out split and a seed per run, and it writes machine readable results (model_testing/notebooks, results/*.json, results/BENCHMARK_REPORT.md; the triage distillation in TriageModel/ and ml/triage/). This section is the evidence behind each choice.

Stage	Candidates	Chosen	Why
Intent and department text	22 models: TF-IDF with five heads, MiniLM frozen with four heads, zero-shot and prototype matching, TextCNN, MiniLM and DistilBERT fine-tuned, plus clustering	MiniLM embedding with a small head	Within 0.005 macro F1 of the best, and its embedding is reused by retrieval and by the triage model
Customer priority (triage)	TF-IDF with logistic regression, MiniLM with logistic regression, distilled multitask head	Distilled multitask head (triage_multitask)	Best urgency F1, 0.08 ms, one embedding shared with the rest of the stage
Router photo	12 models: HOG, colour histogram, raw pixels with PCA, and three CNNs frozen or fine-tuned	EfficientNet-B0 frozen plus logistic regression	0.977 macro F1 for a 0.09 MB head, and no fine-tuning to maintain
Speech	Whisper tiny, base, small and medium; faster-whisper small and medium (CTranslate2 int8)	faster-whisper small int8	Lowest error rate of the sizes that fit, and the fastest real-time factor
Drafting and diagnosis	Ollama Cloud gpt-oss 120b and 20b, Gemini, Groq (five models)	Bound live, per deployment	No retraining: the admin page rebinds them and the plan verifies whichever is bound

3.2.1 Intent and Department Classification

Dataset. The Bitext 27K customer support corpus, de-duplicated, 27 intents, stratified split. **Method.** Every model trains on the same split and is scored on the same held-out set: accuracy, balanced accuracy, macro and weighted F1, top-3 accuracy, training seconds, single-item inference latency and artifact size. Two dummy baselines bound the bottom of the table, and unsupervised clustering (KMeans, agglomerative, GMM, HDBSCAN, LDA) is scored separately with ARI, NMI and Hungarian-matched accuracy.

Model	Family	Accuracy	Macro F1	Inference	Size
tfidf word+char, linear SVC	traditional	0.9992	0.9991	1.4 ms	1.96 MB
MiniLM fine-tuned	deep	0.9969	0.9966	8.3 ms	87 MB
DistilBERT fine-tuned	deep	0.9961	0.9956	8.2 ms	256 MB
MiniLM frozen + linear SVC	embedding	0.9953	0.9946	9.9 ms	0.08 MB
MiniLM frozen + logistic regression	embedding	0.9931	0.9923	9.9 ms	0.04 MB

MiniLM zero-shot label match	embedding	0.7386	0.7281	9.8 ms	n/a
Most frequent class (baseline)	dummy	0.0416	0.0030	n/a	n/a

Nine models clear 0.99 macro F1, so accuracy alone does not decide it. The chosen family is the MiniLM embedding with a small head: it costs 0.04 to 0.08 MB on disk, and the same embedding already has to be computed for retrieval and for the triage features, so the classifier is close to free. A zero-shot model is 26 points worse, which is the measurement that justifies training anything at all.

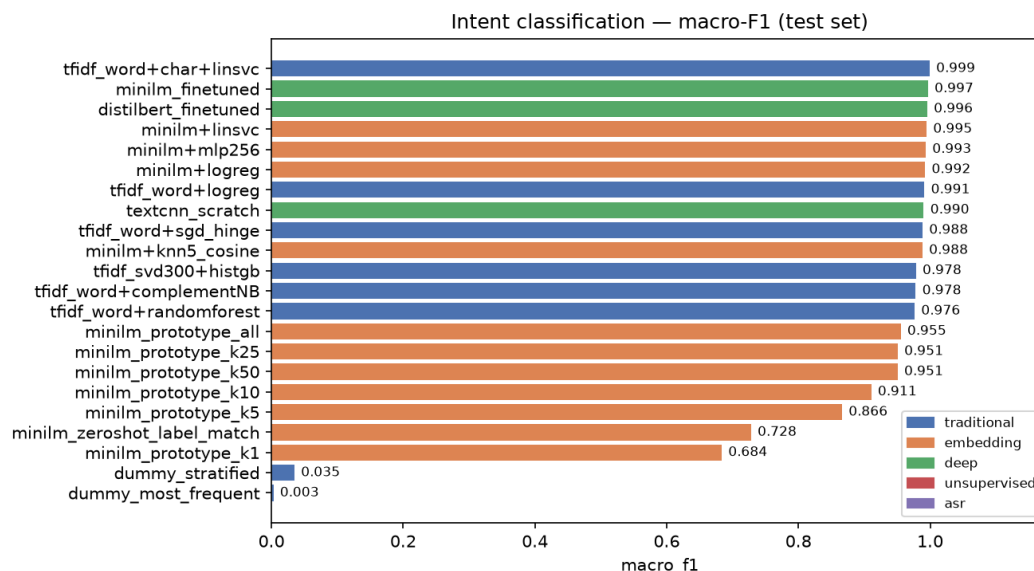


Figure A: Intent classification, macro F1 by model.

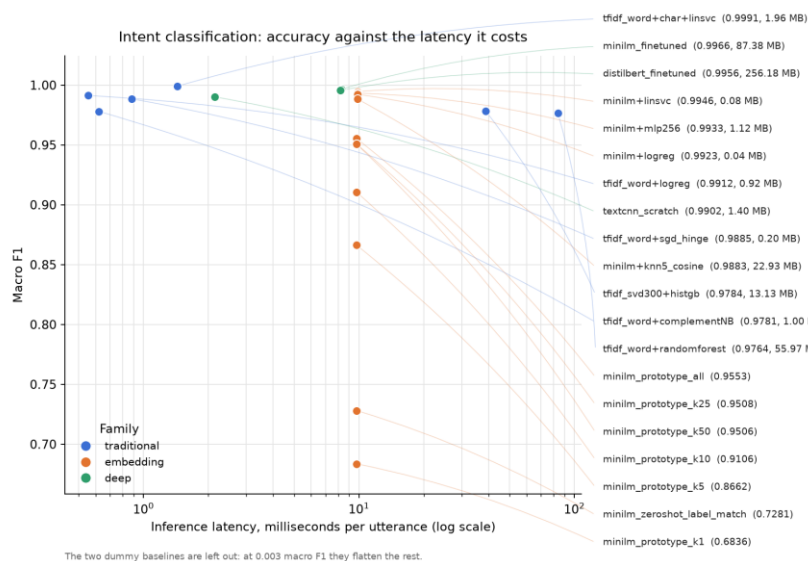


Figure B: Accuracy against inference latency.

Robustness. The same models were scored again on a shifted split (different phrasing distribution) to see which family degrades. The gap is small for the linear models and largest for the fine-tuned transformer, the usual over-fitting signature:

Model	Macro F1 (held out)	Macro F1 (shifted)	Gap
tfidf word+char, linear SVC	0.9991	0.9943	0.005

MiniLM frozen + logistic regression	0.9923	0.9779	0.014
DistilBERT fine-tuned	0.9956	0.9582	0.037
MiniLM zero-shot label match	0.7281	0.7137	0.014

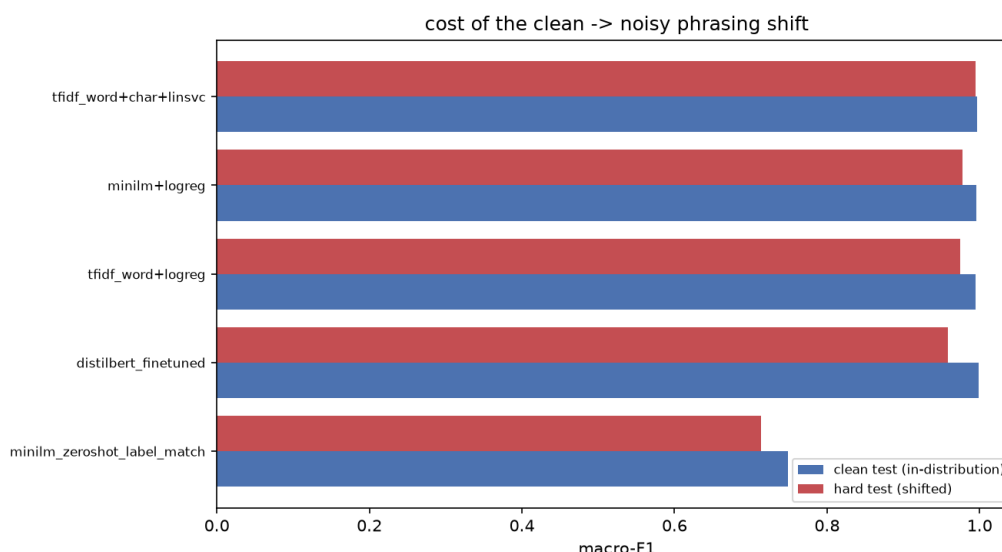


Figure C: What each family loses under distribution shift.

3.2.2 Customer Priority: Distillation from Language Model Labels

No triage-labelled Sri Lankan support data exists, so one was made: about 3,500 records balanced over the 27 intents, labelled once by a language model against the UnifiedTicket schema, with every answer cached by a content hash of prompt, model, temperature and prompt version, so an interrupted run resumes for free and a prompt change correctly invalidates the old labels. A gold slice was re-labelled to measure self-agreement before training. A small multitask head (intent, department, urgency band and a 0 to 100 score) was then distilled from those labels on top of the frozen MiniLM embedding, so the runtime never pays language model cost per ticket:

Model	Intent acc.	Department acc.	Urgency acc.	Urgency F1	Latency
TF-IDF + logistic regression	0.580	0.653	0.667	0.686	0.61 ms
MiniLM + logistic regression	0.713	0.807	0.680	0.701	0.13 ms
Distilled multitask head (ships)	0.607	0.700	0.727	0.724	0.08 ms

The distilled head wins on urgency, which is what the stage is for: the department comes from the routing rules, and the provider side priority comes from the account and network record, not from this model. It is the default binding again after this iteration, with a Groq model selectable for a language model reading of the same message (section 3.1.2).

3.2.3 Router Photo Classification

Dataset. The Roboflow router detection set (v38), whose boxes become a 15-class crop classification problem, cached to disk. **Method.** Classical descriptors and three CNN backbones, each used frozen with a linear head and fine-tuned, on the same split.

Model	Family	Accuracy	Macro F1	Size
EfficientNet-B0 frozen + logistic regression	embedding	0.9704	0.9774	0.09 MB
EfficientNet-B0 fine-tuned	deep	0.9630	0.9728	15.7 MB

ResNet18 frozen + logistic regression	embedding	0.9481	0.9567	0.04 MB
MobileNetV3-small fine-tuned	deep	0.9630	0.9488	6.0 MB
Colour histogram + random forest	traditional	0.8667	0.7723	9.1 MB
HOG + logistic regression	traditional	0.7704	0.6409	0.30 MB
Most frequent class (baseline)	dummy	0.2815	0.0366	n/a

Fine-tuning costs six to seven minutes of training and 170 times the disk for no gain over the frozen backbone with a linear head, so the frozen model ships, exported to ONNX for the image service. The photo is optional evidence in the pipeline: it can add a fact, and it never blocks a reply.

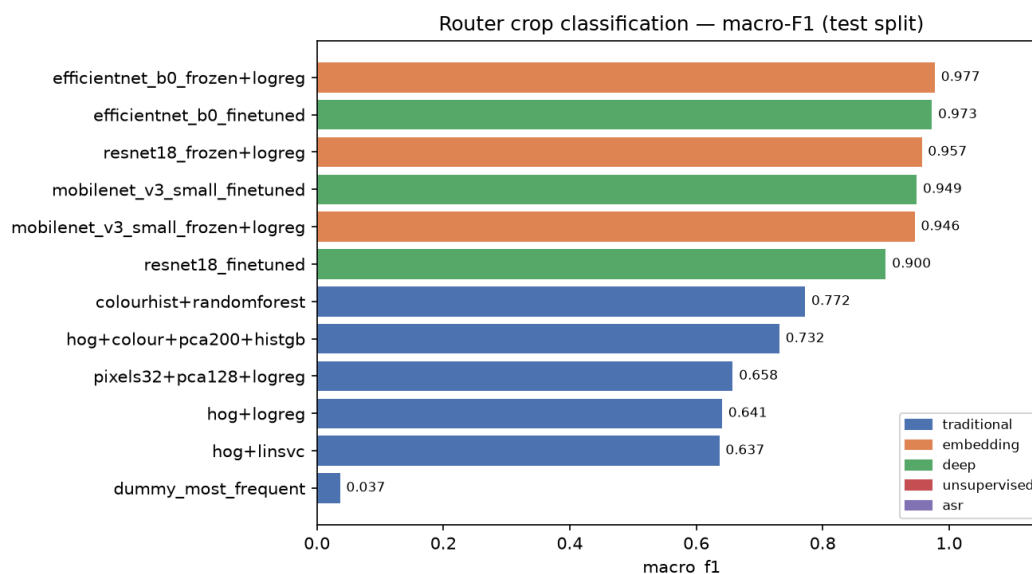


Figure D: Router photo classification, macro F1 by model.

3.2.4 Speech Transcription

Dataset. Eleven real call-centre recordings across seven languages, with cleaned reference transcripts. **Method.** Word and character error rate (duration weighted), match error rate, word information lost, an embedding similarity, the real-time factor and language identification accuracy.

Model	WER	CER	Real-time factor	Language ID
faster-whisper small (int8)	0.390	0.267	0.088	11 of 11
Whisper small	0.430	0.293	0.130	11 of 11
Whisper base	0.597	0.387	0.103	11 of 11
Whisper tiny	0.761	0.591	0.091	11 of 11

faster-whisper small is both the most accurate of the sizes that fit in memory beside the translation model and the fastest, at about one tenth of real time, so it ships. The error rates are high in absolute terms because the recordings are noisy call-centre audio in seven languages; what the pipeline needs from them is the language and the gist, which then go through translation, and the customer's own typed text is always kept beside them. Language identification was correct on every call, which is the property the pipeline depends on.

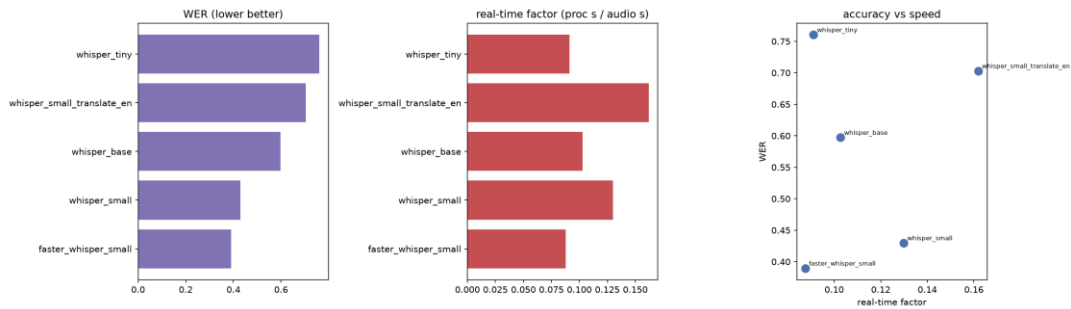


Figure E: Speech benchmark, error rates and real-time factor.

3.2.5 Grounded Response Validation

The drafting design (retrieve the facts first, then let the model write only from them) was validated in a prototype before it became the response service, and that validation still runs offline with no API key: a fake model forces the classifier's decision while the real graph, the real tool transport (MCP over JSON-RPC to a subprocess) and the real database are exercised. It proves the two properties a language model cannot be trusted to hold on its own: that a connectivity question chains **both** the network status and the payment lookup (the naive single-lookup answer blames the router when the real cause is a suspension for non-payment), and that the retrieved facts really appear in the generation prompt.

Result: 56 checks passed across 5 scenarios, none failed. The same rule is enforced at runtime by the grounding bundle and the compliance guard of section 3.1.2, which is why the headline case cites the balance and the outage instead of guessing.

```
D:\DSEP22\response-prototype\.venv\Lib\site-packages\pydantic_settings\sources\utils.py:47:
IncompleteFieldDefinitionWarning: Field 'lifespan' has an incomplete definition: its annotation contains an
unresolved forward reference, so settings sources may fail to correctly resolve its value. Call
`model_rebuild()` on the model where the field is defined, once all the referenced types are defined.
  warnings.warn(
Processing request of type ListToolsRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
Processing request of type CallToolRequest
MCP tools loaded: ['get_customer_by_id', 'get_payment_status', 'get_network_access_status',
'get_plan_details']

PASS server exposes 4 tools
... 58 lines omitted ...
PASS generate node ran once
PASS grounding block present in prompt
PASS fact found=False embedded in grounding prompt
PASS answer written to state

KEY ASSERTION - connectivity multi-tool chain:
PASS connectivity chained network AND payment lookups

=====
ALL CHECKS PASSED
```

Listing 12: Grounded response validation, offline, with the real tool transport.

4. Deliverables

The test effort delivers the following, all in the repository or produced by one command:

- Test logs and JUnit XML for every run (reports/<date>/, or the GitHub Actions artifact); the logs this report quotes are kept beside it in docs/test-report/evidence/logs.
- User interface screenshots of every page, on a phone and on a desktop browser, kept by the browser suites on every run.
- Load test summaries from k6, for the isolated write path and for production.
- The production monitoring stack and its Grafana dashboard template (infra/monitoring).
- The model selection benchmarks, their results and figures (model_testing/results).
- This report, generated from a run's evidence (docs/test-report).

4.1 Test Evaluation Summaries

Summary of the final run, generated by scripts/test-plan.sh on the isolated stack:

Step	Executed	Verdict	Time
Unit and contract	131 tests	Pass	18s
Stack up and smoke	1 scenario	Pass	59s
System (all techniques)	51 tests	Pass	102s
Browser, desktop and mobile	16 tests	Pass	90s
Headline scenario	1 scenario	Pass	32s
Live model swap	1 scenario	Pass	1s
Load, write path	602 requests	Pass	62s

Field	Value
Date	20 September 2026
Run by	DSEP Group 22, scripts/test-plan.sh
Environment	Full stack (16 services, both models) on an isolated Postgres; Windows 11, Docker Desktop, 16 cores, 7.9 GB for Docker
Models on the path	Groq qwen/qwen3.8-27b (triage), Ollama Cloud gpt-oss 120b and 20b (drafting, diagnosis), NLLB-200, faster-whisper
Test cases executed	200
Passed	200
Failed	0
Skipped	0
Pass percentage	100.0 percent
Comments	Every defect found by this iteration's runs is listed below with its fix, except D13, which is open.

Defects found in this iteration (D1 to D9 were found and fixed in the previous run of 19 September):

#	Found by	Defect	Cause	Fix
D10	Configuration (clean install)	The inquiry service never starts on an empty database	A backfill UPDATE read inquiry.message before the CREATE TABLE later in the same script	Backfill moved after the table; fixed
D11	Configuration (clean install)	Seeding fails on any database without TLS	The seed forced TLS regardless of the connection string	TLS only when the connection string asks for it; fixed

D12	User interface	Signing out of the customer area lands on sign-in instead of the website	The role gate saw the session end and redirected before the sign-out navigation landed	One sign-out helper that holds the gate while leaving; fixed
D13	Function (reading the reply)	A Sinhala reply split a conjunct: "මධ්‍යම" for "මධ්‍යම"	The conjunct repair only joins across a space after a bare consonant, to avoid D2	Open: needs a Sinhala word list to tell a split conjunct from a word boundary
D14	Function (Groq triage)	Every second Groq call failed with 429	No max_tokens: Groq reserved the model's whole window against an 8,000 token a minute limit	max_tokens and temperature set per binding; fixed
D15	User interface (mobile)	Plans page empty after every deploy until the cache expired	The page was prerendered at image build time, when no gateway exists	Rendered per request with a one minute data cache; fixed
D16	User interface (mobile)	Four pages scroll sideways on a phone; the customer menu hides four items	Header display class overridden, grid children without min-width, fixed widths	See 3.1.3; fixed
D17	System (function)	A new case could fall past the console's 100 row cap after a burst of urgent ones	The history tab still sorted by priority before time	The All tab is ordered by time alone; fixed
D18	System (function)	Drafts slower than 30 s were lost, although drafting is allowed 60 s	The orchestrator's shared HTTP client kept its own 30 s read timeout	Each stage call times out at its own budget; fixed
D19	Scenario	A reply quoting an incident id or an ISO date was held as a phone number	The detector matched any nine digits and hyphens	A phone is ten or more digits standing alone; fixed
D20	Scenario	Markdown reached the customer (bold markers around an amount)	The scrub ran per streamed chunk, so a marker split across two survived	plain_text() strips markdown after the stream is joined; fixed
D21	Test stack	migrate failed against a brand new database volume	The health check used the Unix socket, which the temporary init server also answers	Health checks go over TCP; fixed
D22	Monitoring (production)	cAdvisor reported no containers	Versions before v0.55 cannot read Docker 29's containerd image store	Pinned to v0.55.1; fixed
D23	Monitoring (production)	The model map showed the distilled triage model as Not answering	A binding that runs inside its own service has no endpoint to ping, and the probe returned unknown	The probe reports where the model runs; fixed
D24	Production (environment)	Drafting and diagnosis cannot reach any language model on the server	The Ollama Cloud key is rejected (401), Gemini refuses the server's region ("User location is not supported"), and the Groq key was missing	GROQ_API_KEY set as a repository secret; after the deploy, drafting answers in 755 ms and diagnosis in 558 ms from the server; fixed

```

$ bash scripts/test-all.sh
Resolved 106 packages in 8ms
Audited 74 packages in 0.43ms
All checks passed!
..... [100%]
5 passed in 0.09s
..... [100%]
===== warnings summary =====
services/gateway/tests/test_jwks.py::test_hs256_alg_confusion_rejected
  D:\DSEP22\v3\.venv\Lib\site-packages\jwt\api_jwt.py:149: InsecureKeyLengthWarning: The HMAC key is 6 bytes
long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
    return self._jws.encode(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
9 passed, 1 warning in 0.11s
..... [100%]
20 passed in 0.63s
..... [100%]
5 passed in 0.02s
..... [100%]
===== warnings summary =====
app\main.py:80
  D:\DSEP22\v3\services\trriage\app\main.py:80: DeprecationWarning:
... 46 lines omitted ...
    type: 'test'
    ...
1..2
# tests 2
# suites 0
# pass 2
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 217.914
test-all OK

```

Listing 11: Unit, contract and architecture tests, lint and typecheck.

4.2 Reporting on Test Coverage

Coverage is reported by requirement rather than by line, because the risk in this product is a missing conversation or a leaked row, not a missed branch.

Technique	Where	Tests
Unit and contract	packages, services/*/tests, tests/architecture	131 plus Node tests and module self-checks
Data and database integrity	tests/system/test_data_integrity.py	8
Function	tests/system/test_functional.py, 2 scenario scripts	6 + 2
Security and access control	tests/system/test_access_control.py	15
Performance	tests/system/test_performance.py	8
Configuration	tests/system/test_configuration.py	10
Failover and recovery	tests/system/test_resilience.py	4
User interface	web/e2e (desktop and mobile)	16
Load	load/ack.js, load/browse.js	2 scenarios, 7 thresholds
Model selection	model testing/notebooks, TriageModel, ml/triage	49 models over 4 tasks
Grounded response validation	response-prototype/test_offline.py	56 checks, 5 scenarios

How to reproduce any number in this report:

- `bash scripts/test-plan.sh` on any machine with Docker: the whole plan on a throwaway database, evidence in `reports/<date>/`.
- GitHub, Actions, test-plan, Run workflow: the same on a GitHub runner, evidence as a downloadable artifact.
- `SCRIPT=load/browse.js LANKA_URL=https://172-197-200-110.sslip.io bash scripts/load.sh`: the read-only production load test.
- The logs, JUnit XML and k6 summaries this report quotes are kept in `docs/test-report/evidence`, so every number here can be checked against them.

Operational monitoring. Coverage does not stop at release. Production now runs Prometheus and Grafana beside the stack (`compose.monitoring.yaml`, deployed by the same pipeline). The provisioned dashboard shows the services that answered their last health probe, the public URL checked through DNS and TLS, CPU, memory, disk, every container's CPU and memory, request rates, status codes and latency percentiles at the edge, and the certificate expiry. Alert rules cover a service or the site going down, the certificate, memory, disk, CPU and error rate.



Figure 12: The production dashboard (Grafana at `/grafana`).

5. Risks, Dependencies, Assumptions, and Constraints

Risk	Mitigation Strategy	Contingency (Risk is realized)
External model capacity limits the pipeline	• Measured: about eight drafts a minute on the current Ollama Cloud account; Groq triage 8,000 tokens a minute. • Every model stage has a fallback or a budget, so a limit degrades a draft, never a message.	• Move drafting to a paid tier or to Groq. • Add a draft queue in arrival order.

Risk	Mitigation Strategy	Contingency (Risk is realized)
Model provider limits and regional blocks	- Groq's free tier has per-minute token and weekly request limits; triage falls back to the distilled model, which is also the default, so a limit costs nothing on the hot path. - Every model binding is swappable live from the admin page, with no restart and no deploy.	- Move the drafting account to a paid tier. - Gemini answers 'User location is not supported' from the server's region, so it cannot be the fallback for this deployment; keep Ollama Cloud or a second Groq key for that.
Seeded demo passwords are public and live on the public site	Documented; changing the SEED secrets does not change existing accounts.	Reset the seeded accounts' passwords, or remove them from production.
Tests running against production data	compose.test.yaml isolates every write test; only read-only load runs on production.	The shared database has point-in-time restore on Neon.
Model output is not deterministic	Tests assert facts (digits, script, citations), never wording; triage runs at temperature 0.	A person reviews the Sinhala and Tamil wording; D13 shows why.
Measurements from a development machine	Every figure states where it was taken; production is measured separately, read only.	Repeat the load test on the VPS when traffic grows.
Chromium only	Desktop and phone profiles cover layout and touch; engines differ little for this app.	Add WebKit and Firefox projects to Playwright.

Dependencies. Docker; the GHCR images; API keys for Groq, Ollama Cloud and Gemini (without them triage falls back to its own model and drafting to the offline stub); the NLLB and Whisper weights inside the images; the Neon project for production.

Assumptions. The seeded personas exist (the test database seeds them itself); the machine can hold both models resident (about 4 GB).

Constraints. One iteration and one team; no dedicated test hardware, so the load test of the write path runs on a development machine and production is only tested read only; the Groq free tier.

6. References

- [1] IBM, "Rational Unified Process: Master Test Plan template." Provided by the course. Accessed: Sep. 20, 2026.
- [2] Microsoft, "Playwright: fast and reliable end-to-end testing." [Online]. Available: <https://playwright.dev/> (Accessed: Sep. 20, 2026).
- [3] Grafana Labs, "k6: load testing for engineering teams." [Online]. Available: <https://k6.io/> (Accessed: Sep. 20, 2026).
- [4] pytest developers, "pytest documentation." [Online]. Available: <https://docs.pytest.org/> (Accessed: Sep. 20, 2026).
- [5] Prometheus authors, "Prometheus: monitoring system and time series database." [Online]. Available: <https://prometheus.io/> (Accessed: Sep. 20, 2026).
- [6] Grafana Labs, "Grafana documentation: provisioning." [Online]. Available: <https://grafana.com/docs/grafana/latest/administration/provisioning/> (Accessed: Sep. 20, 2026).

- [7] Google, "cAdvisor: container advisor." [Online]. Available: <https://github.com/google/cadvisor> (Accessed: Sep. 20, 2026).
- [8] Groq, "GroqCloud API reference and rate limits." [Online]. Available: <https://console.groq.com/docs/rate-limits> (Accessed: Sep. 20, 2026).
- [9] Gitleaks, "Gitleaks: find secrets with Gitleaks." [Online]. Available: <https://github.com/gitleaks/gitleaks> (Accessed: Sep. 20, 2026).
- [10] NLLB Team et al., "No Language Left Behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.
- [11] A. Radford et al., "Robust speech recognition via large-scale weak supervision," arXiv:2212.04356, 2022.
- [12] Lanka Link v3 repository, docs/TEST-PLAN.md and docs/DEPLOYMENT.md. [Online]. Available: <https://github.com/DSEP-G22/v3> (Accessed: Sep. 20, 2026).